

Computer Graphics: Lecture 17

Spheres, Cube-maps, and Skyboxes

Slide Deck © Grant Williams, 2026, License: [CC-BY-SA 4.0](https://creativecommons.org/licenses/by-sa/4.0/)

Welcome back!

Last time we learned the whole Phong model

We learned how to draw not only directional lights, but point lights as well. [what's the difference?]

We also learned about storing structs in WGSL, how it can help organize the software design of our shaders but how it also requires understanding *alignment*. [how does alignment work?]

This time

We're going to learn about generating spheres.

Really? That's all?

That's all...but it's a bit more challenging than you think.

We're also going to learn a new texturing technique: the cube map.

Finally: no more duplicating the UV coordinates on a cube.

Cube mapping isn't just useful for texture mapping cubes: it's great for spheres, too, and it even enables some cool features such as skyboxes and environment maps (shiny textures that reflect the environment).

Spheres

Okay, this is a geometry pop quiz. If you had to generate a sphere mesh *right now*, what would you do?

[Let's think about this as a class.]

Circles

Chances are, learning to generate a circle will help us understand how to generate a sphere.

So let's start with generating a circle.

Ultimately, our video card wants to draw triangles. It is technically possible to use fancy tessellation shaders to generate a circle that is visually perfect (i.e., at a given resolution it is impossible to tell it from an ideal circle).

But let's not worry about that now. Let's just generate a regular polygon with a lot of sides that we will interpret as a circle if it has enough sides.

Circles (2)

The most straightforward way to generate a circle is to first decide how many sides we want. Say, 100. Then, plot that many points in the right place.

```
const verts = []; // vertex data: x,y coordinates
const nPoints = 100;

for (let point = 0; point < nPoints; point++) {
    ...
}
```

So...[how do we know where a point goes? What are its coordinates?]

Circles (3)

These points are going to go around the circle. Over the course of this process, the points will complete a single turn.

That means, the first point will start at 0 turns, the second will then be at one hundredth of a turn, the third will be two hundredths, etc.

So, we can determine the number of turns like this:

```
for (let point = 0; point < nPoints; point++) {  
    const turns = point / nPoints;  
}
```

But that's not enough. How do we turn *turns* into XY coordinates?

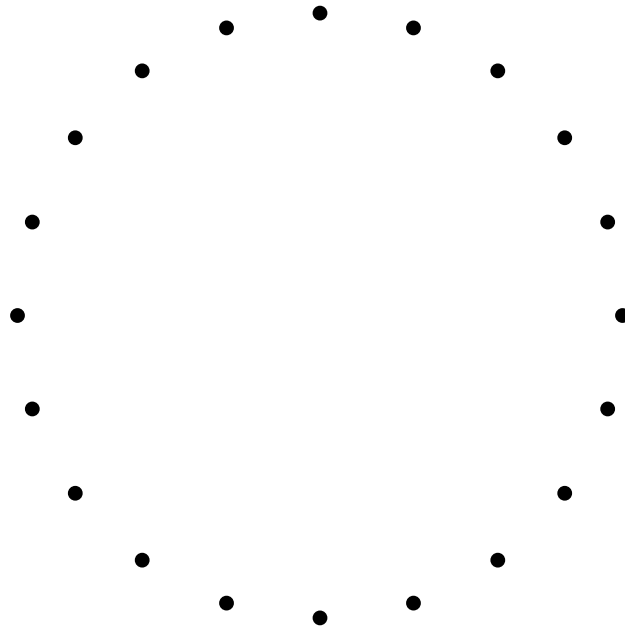
Circles (4)

First we need to convert them into angles in radians. Then we can use sine and cosine on the radians:

```
for (let point = 0; point < nPoints; point++) {  
  const turns = point / nPoints;  
  const rads = turns * TAU;  
  const x = Math.cos(rads);  
  const y = Math.sin(rads);  
  verts.push(x, y);  
}
```


Circles (5)

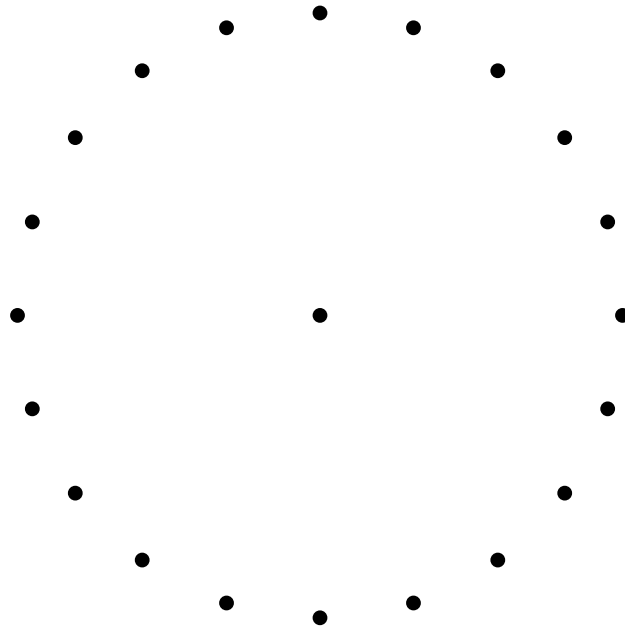
I coded the software renderer in my slides software to follow this algorithm using 20 vertices. It's pretty close to a circle.



Circles (6)

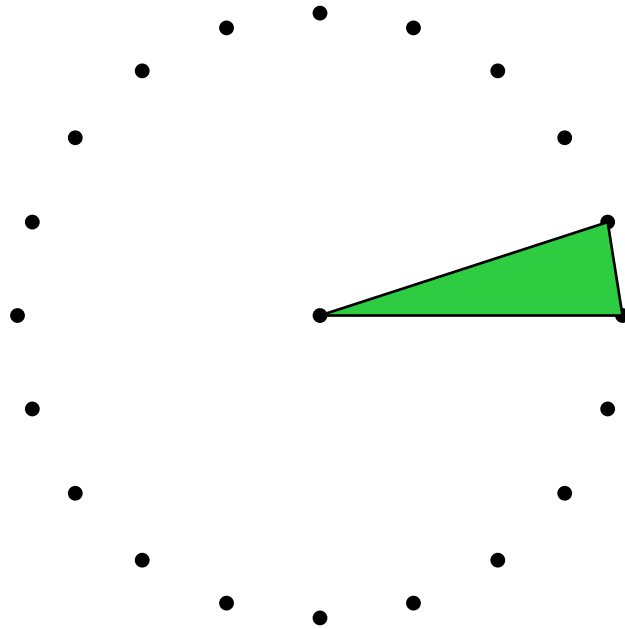
But a mesh isn't just a collection of points. It's also a collection of faces! (i.e., triangles). So how can we stitch our mesh together into triangles?

You might think we need to put a vertex in the center like this:



Circles (7)

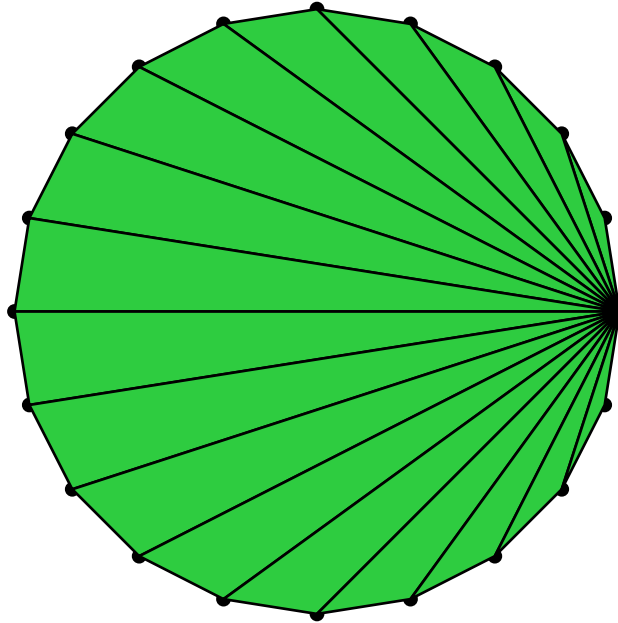
Then, we can stitch together a circle by creating triangles that pass through the center...



I.e., do this for all 20 slices.

Circles (8)

However, there is another way, where we just use one of the points on the side as the start of the triangle.



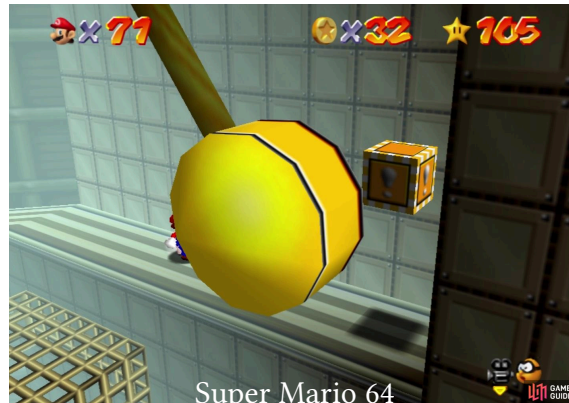
This saves a whole vertex 🤪. Despite looking weird, it's just as good.

Triangle fans

There's a name for this kind of topology: a triangle fan.

It was built into legacy graphics APIs like OpenGL.

The first index in the strip would be the shared vertex, and then the other indices would be specified in the winding order. [0, 1, 2, 3, ...]



The pendulum's disc in Tick tock clock was a triangle fan.

Triangle fans (2)

Triangle fans were useful because you could generate any convex polygon from them. As long as the polygon was convex, you could use any point as the “center” point.

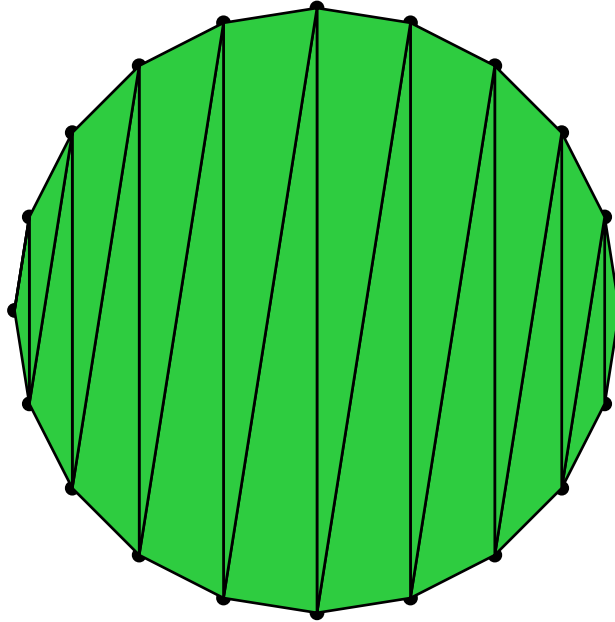
The screenshot before used the west most point in the disc.

However, WebGPU does not support triangle fans.

The reason is that it’s actually possible to use a triangle strip anywhere you would use a triangle fan, with the same number of indices.

[can anyone think of how to do it?]

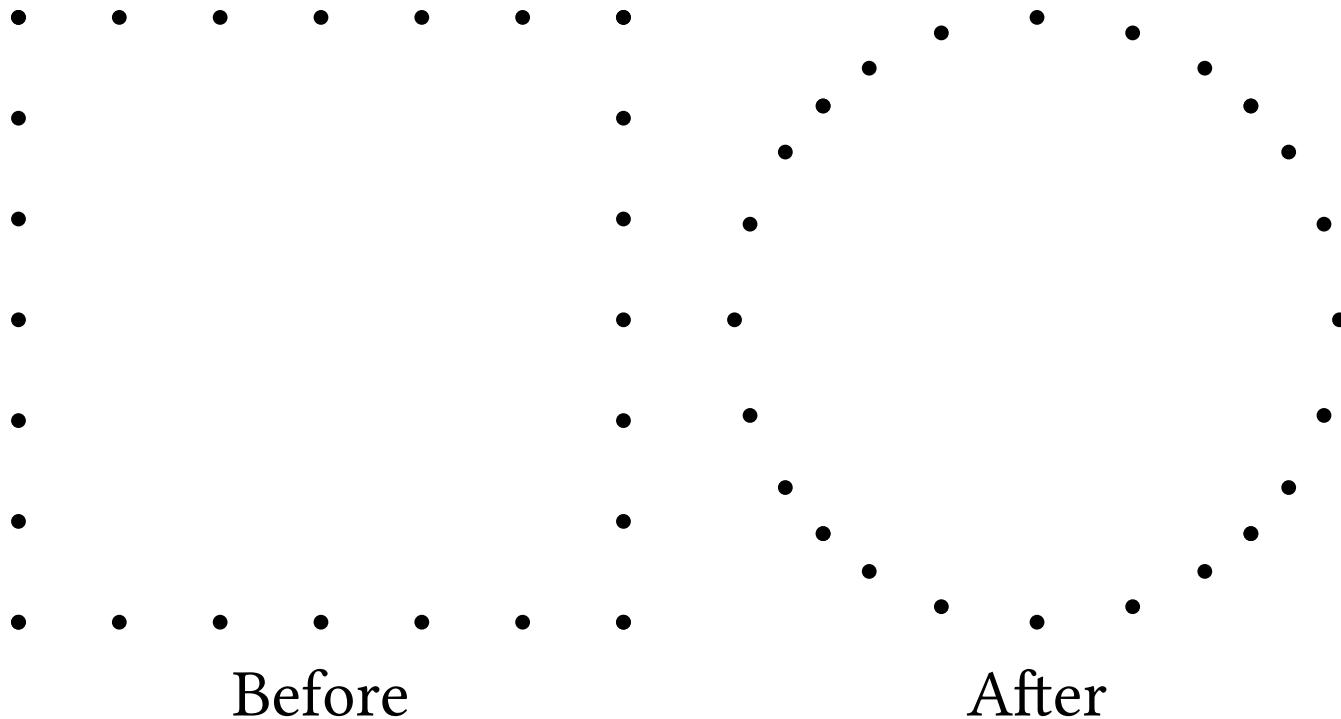
Triangle fans (3)



This saves a whole vertex and requires exactly as many indices as vertices! Just as good as a fan. [0, 1, 19, 2, 18, ...]

Circles: more ways

There are actually several ways to generate circles. We can even start with a square and smoosh the points into a circle by normalizing them:



Circles: more ways

It works because a circle is a shape where every point on it has the same distance from the center. Assuming the center is 0, if we normalize all the points, the circle will have a radius of 1.

It's also not ideal. It ends up with the point distribution being uneven, as you can maybe see. We'd like the vertices to be spread more evenly.

However, it turns out that this is a great way of creating a sphere. It's called a cube sphere, and it's the *second* sphere algorithm we're going to learn today. Keep it in mind!

But first...

Questions?

Now spheres

Let's try to extend our algorithm for making circles.

Imagine that we made the equator of a large sphere that way.

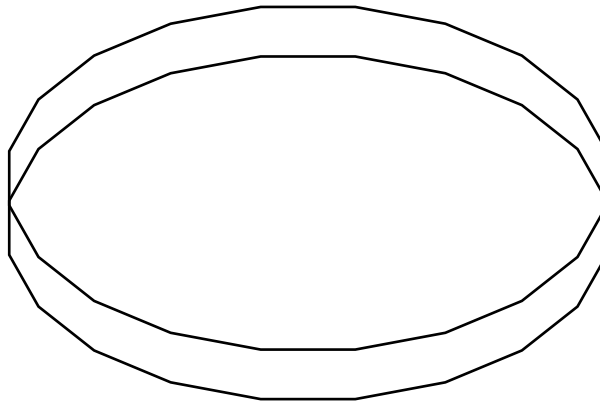
Could we imagine how to stack more rings around the equator?

This kind of sphere is called a **UV-sphere**, because it's easy to compute the UV coordinates of its vertices.

[But how do we build one?]

UV-spheres

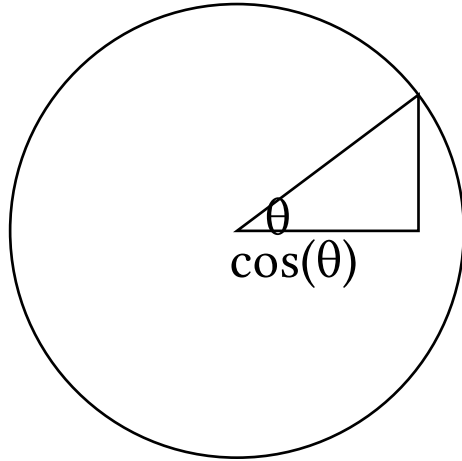
UV spheres are built out of rings, like the ones we generated earlier. Only, we don't fill these rings into circles. We'll see how to tessellate them later.



Here are two rings. Assume the bottom is the equator. How do we shrink the top one to make it start to curve like the top of a sphere?

UV-spheres (2)

Let's imagine a vertical cross section. What we want is the cosine of the angle between two layers in the sphere:

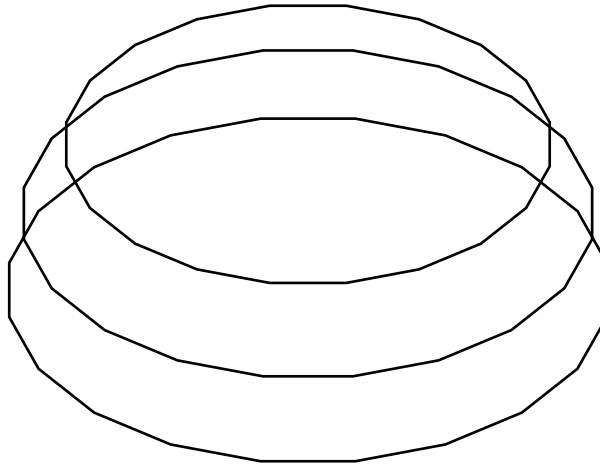


We can choose the angle of the height of the ring, or generate it. Either way the cosine of that angle is the radius of the ring.

UV-spheres (3)

And the y value is the sine of the angle.

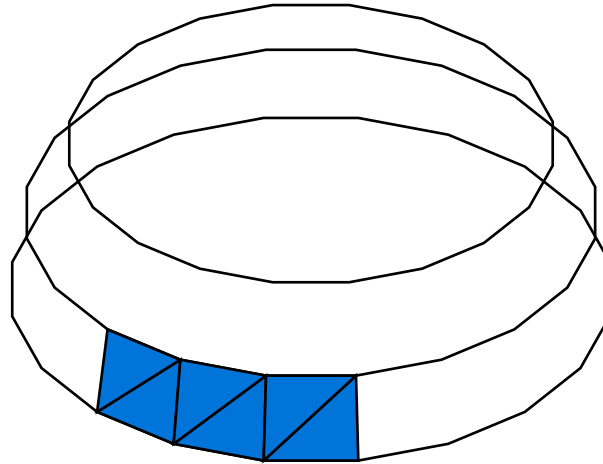
Let's stack 3 rings for good measure:



Hopefully we see the upper hemisphere start to form. But how do we form the triangles?

UV-spheres (4)

We pair adjacent rings together into strips. Then, we can draw a triangle strip all around the sphere. Here are some triangles: this pattern repeats:



We can draw the strip several ways. For example, if the points go clockwise: $[0, \text{pointsPerRing}, 1, \text{pointsPerRing} + 1, 2, \dots]$

UV-spheres (5)

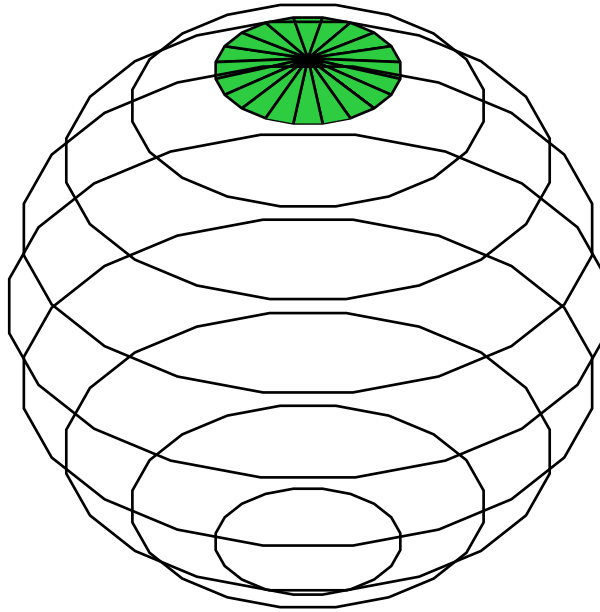
But what happens when we get to the top?

That's where it gets gnarly: you might think we'd put a single vertex at the top but we can't. Remember that these vertices need to have not only XYZ coordinates, but also UV.

If the top ring were connected to a single point, they'd all have to share the same UV coordinate, which would warp the texture.

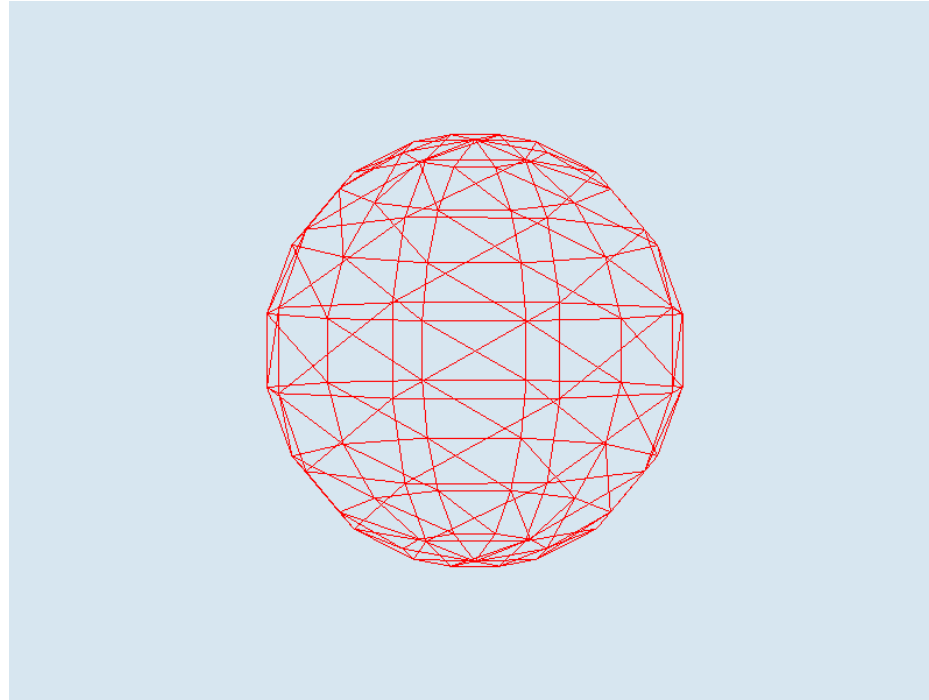
Instead we create an infinitely small ring at the very top. This allows each point in this ring to have a different UV coordinate, but also means that the top of the mesh is sealed (we say it's a **watertight** mesh)

UV-spheres (6)



Following the same triangle strip, half of our triangles are degenerate, because two adjacent points on the top ring are infinitely close together. You can special-case this if you want a slightly more optimized mesh.

Finished UV-sphere



A wireframe UV-sphere generated using the algorithm we've discussed.
See `sample14` for code.

UV-spheres (7)

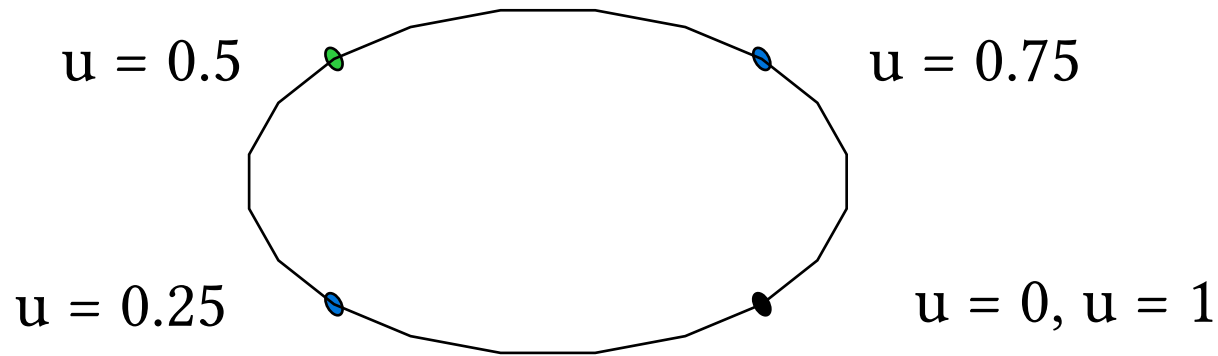
So far we've talked about how to generate rings out of vertices with known XYZ coordinates, and how to stitch those rings together into layers using indices. But what about UV coordinates?

The main point of UV spheres is that we want to wrap a single texture around them. So we want the U coordinate to increase as it goes around a latitude, and we want the V coordinate to increase as it goes toward the bottom pole.

So, [how can we compute UV coordinates?]

UV-spheres (8)

For a given ring, we can make the U coordinate of the first vertex 0. We want the last vertex to have a U coordinate of 1, and for it *to overlap the first vertex*.



In general, the i th point will have a U coordinate of $i / (\text{numberOfPoints} - 1)$, where numberOfPoints includes the last point that overlaps the first one.

UV-spheres (9)

But what about the V coordinate. We want the V coordinate to increase as the y value goes *down* (because remember, a V of 0 means the top of the texture).

[Any ideas?]

UV-spheres (10)

The simplest approach is, for ring number i , to just let $v = i / (\text{num_rings} - 1)$. This is assuming you're building the mesh from top to bottom.¹

If you're going from bottom to top (so $i = \text{numRings} - 1$ at the top ring), take the complement: $v = 1 - i / (\text{num_rings} - 1)$.

Alternatively, you can use an already-computed y component: $v = y * 0.5 + 0.5$. y normally ranges from -1 to 1 , so this squeezes it into the range 0 to 1 .

¹some textures are distorted so that they are “squeezed” around the poles to counteract the fact that the y distance between rings is not constant. You can use the vertical angle of the ring and scale it to the range $[0, 1]$ for these textures.

Questions?

UV-sphere problems

UV-spheres are a very simple way to draw a sphere with a texture painted over it. sample14 shows how that can be used to draw a globe (the UV-sphere is on the left).

They aren't an ideal topology, however, and they are typically only used when we want to wrap a cylindrical texture.

[Can anyone think of a problem with them?]

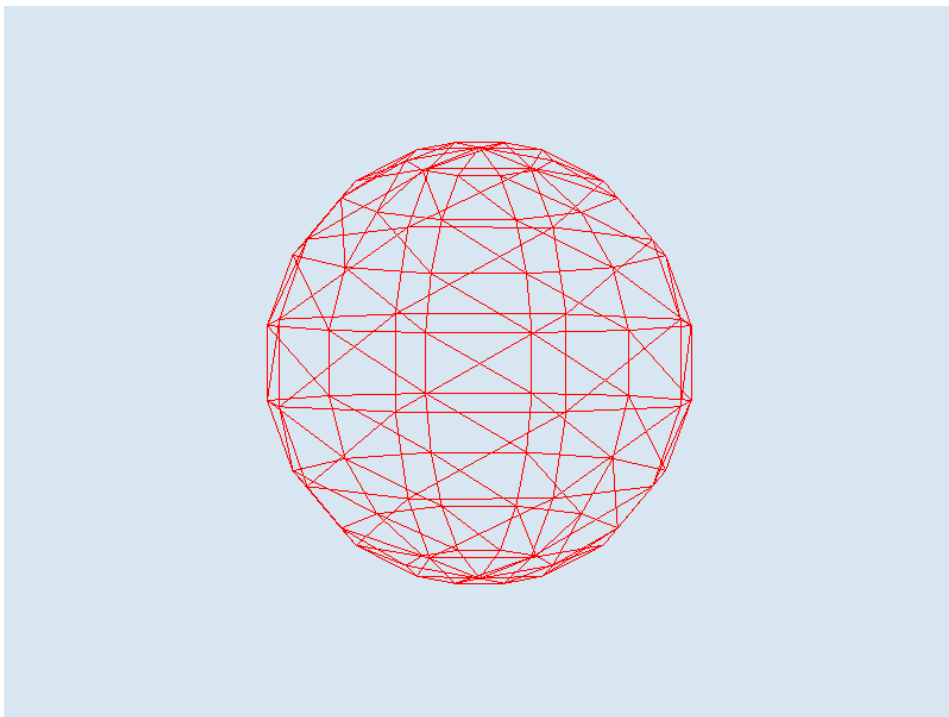
UV-sphere problems (2)

There are really two issues with UV-spheres:

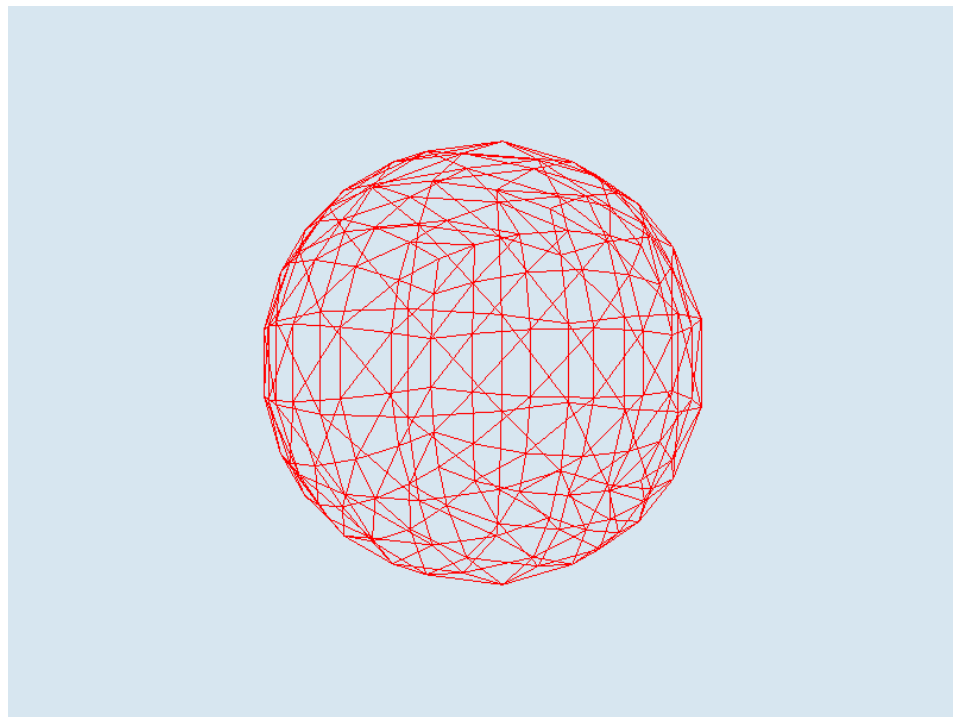
1. Poor point regularity: there are tons of points at the poles compared to the equator.
2. Texture distortion: as a result, the sphere has the lowest detail at the equator, where detail is often the most important!

To illustrate this, in the next slide, compare the wireframe of the UV-sphere to the wireframe of the next type of sphere we're going to learn...

UV-sphere vs Cube-sphere



A uv-sphere. Notice that the center faces are much larger than the ones near the top.



It's a little hard to tell, but this sphere's faces are much more evenly-sized.

Introducing cube-spheres

Let's learn how to make **cube-spheres** next. They have several advantages over UV-spheres:

1. They have better vertex distribution¹
2. They can be more evenly textured

First, let's see how to build a cube-sphere. Then, we'll learn a cool new texturing technique that is ideal for them called cube-mapping.²

¹although still not perfect: that property belongs to **icospheres**

²You can also use this technique on UV-spheres, but the texture will be sampled more unevenly, possibly causing minor distortion.

Building a cube sphere

Now it's time to build one.

Remember earlier in the lecture when we built a circle by starting with a square and then squishing it into a circle.

But this time it will be *good* instead of *bad*.

Wait...won't there still be distortion? Won't there be more points away from the faces of the cube?

Yes, but we're going to tolerate that, because it won't be as bad as the UV-sphere and it will texture better.

Building a cube sphere (2)

Let's start with the front face.

We want to, given a number of subdivisions, create a grid of points in a square shape. We can use a basic 2D for loop pair for this.

For each point, we want to normalize it so that we pull it into a sphere.

That's it. If we do that for a whole square, we will have one-sixth of a cube sphere.

Building a cube sphere (3)

```
const spanVerts = nFaceSubdivs + 1;
// front face, in the Z+ direction (right-handed)
for (let row = 0; row < spanVerts; row++) {
    for (let col = 0; col < spanVerts; col++) {
        const y = 2 * row / nFaceSubdivs - 1;
        const x = 2 * col / nFaceSubdivs - 1;
        const l = Math.sqrt(x * x + y * y + 1); // length
        // quiz: where does the 1 come from in the sqrt?
        // push the normalized coordinate:
        verts.push(x / l, y / l, 1 / l);
    }
}
```

Building a cube sphere (4)

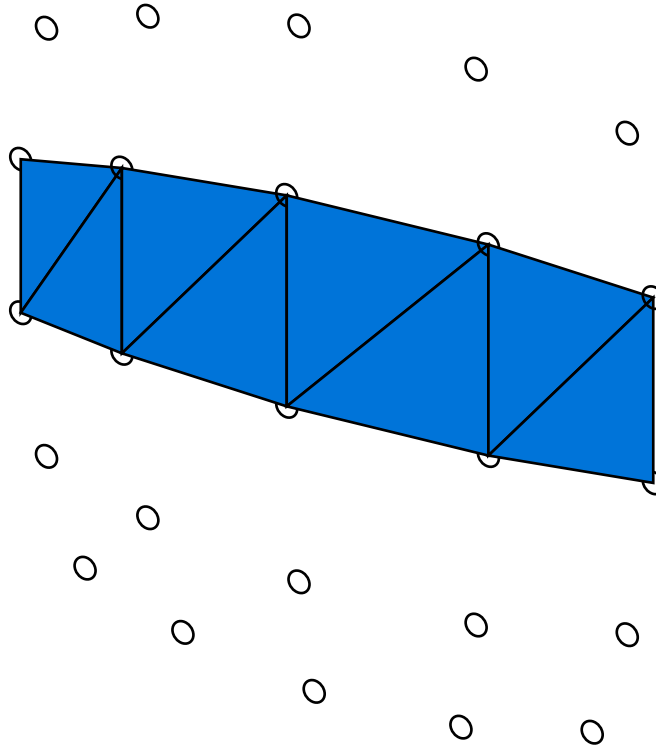
We do this for all six faces. The only wrinkle is making sure they all face *outward* (be mindful of winding).

But what about indices?

Believe it or not, that's actually easier than it was for the UV-sphere. We have a grid of points for each face. We just need to build a triangle strip.

The simplest way to do this is to build a “ribbon” going along each row...

Building a cube sphere (5)



Here is one such ribbon. I've rotated the image in 3D slightly so you can see how it bends inward.

Building a cube sphere (6)

So, a triangle strip might look like this:

```
[rowStart + vertsPerRow, rowStart, rowStart + vertsPerRow +  
1, rowStart + 1, etc.
```

And what happens at the end of the row?

```
..., 0xFFFFFFFF]
```

That's the restart index. It tells the GPU to cut the strip.

So, we draw a perfectly normal grid of vertices, stitched into a triangle strip, except we also shrink the points until they are on the surface of a sphere. We do this for each face. That's a cube sphere.

Building a cube sphere...UVs?

So now you might wonder about UV coordinates.

They're pretty straightforward if we want them to be: we can have each face sample a square in UV space. Let's say we wanted every face to have the same texture.

Then the top left corner of each face would have coordinates $(0, 0)$, and the bottom right would have coordinates $(1, 1)$.

But what if we *don't* want each face to have the same texture? Well, the techniques we've learned so far are somewhat suboptimal...

Cube sphere UVs (2)

1. We can squeeze all 6 faces into a single texture. This works but it wastes a lot of memory and means we need a much bigger texture total. For example, I did this with the [earth cube-sphere](#) in sample14, and it required a 2048-by-1536 texture, but the sides were only 512-by-512, and only half of the texture is used.